

Python 12

Elias Wachmann

2025

Content

1. What is object oriented programming?
2. Python classes
3. Dunder-methods
4. Inheritance
5. Wrap up + What's next?

What is object oriented programming?

Intro to object oriented programming

How do we classify things in the real world?

We group them by their properties and their behaviour.

An object like a planet is can be described by its properties like mass, radius, position, velocity, etc.

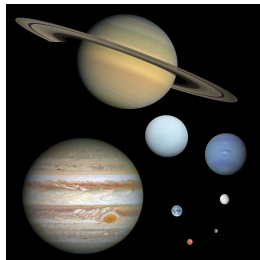
We can also describe its behaviour like how it moves around the sun.

What are classes?

Classes are a way to abstract real world things into categories.

Not all planets are the same, but they can be described by the same properties and behaviour.

We can use a class that describes the properties and behaviour of a planet (an object).



https://upload.wikimedia.org/wikipedia/commons/c/cf/Planet_collage_to_scale.jpg

Describe objects within a class

How can we distinguish between different planets now?

Earth: $m = 5.97 \times 10^{24} \text{ kg}$, $r = 6371 \text{ km}$, ...

Mars: $m = 6.41 \times 10^{23} \text{ kg}$, $r = 3389.5 \text{ km}$, ...

Jupiter: $m = 1.90 \times 10^{27} \text{ kg}$, $r = 69911 \text{ km}$, ...

Python classes

Let's define a Planet class

In python we can define a class like this:

```
1 class Planet():
2     def __init__(self, name_in, radius_in,
3         mass_in):
4         self.name = name_in
5         self.radius = radius_in
6         self.mass = mass_in
```

The class-keyword tells python that we want to define a class.

What is the `__init__` function?

The `__init__` function is called when we create a new object of the class.

Objects are therefore concrete instances of a class.

In our example the Planet class takes the arguments mass and radius and stores them as attributes of the object.

How can we access these attributes and what is the self-keyword?

Python's self keyword

The self keyword is a reference to the object itself.

```
1     def __init__(self, name_in, radius_in,  
2         mass_in):  
3         self.name = name_in  
4         self.radius = radius_in  
5         self.mass = mass_in
```

This means: If you want to create an attribute of a class, you simply write **self.variable_name** self also

has to be passed in as the first argument to every function defined inside the class!

Methods – class functions

Methods are functions inside a class. They can only be accessed by objects of that class and `self` is their first argument.

```
1 class Planet():
2     def __init__(self, name_in, radius_in,
3         mass_in):
4         self.name = name_in
5         self.radius = radius_in
6         self.mass = mass_in
7
8     def getData(self):
9         return {"name": self.name, "radius":
10             self.radius, "mass": self.mass}
```

Let's create a Planet

Finally let's create a planet object:

```
1 earth = Planet("Earth", 6371, 5.972e24)
2 mars = Planet("Mars", 3389, 6.39e23)
```

Just pass in the initial arguments just like you would for functions.

```
1 earth_data = earth.getData()
2 print(earth_data["name"])    # Earth
```

We can now call our defined class method on objects.

Modify objects

Certainly a method can modify attributes of an object like in the following example the name.

```
1     def changeName(self, newname):  
2         self.name = newname  
  
1 earth.changeName("Earth2.0")  
2 earth_data = earth.getData()  
3 print(earth_data["name"])    # Earth2.0
```

This only changes the name of the ,earth'-object while the ,mars'-object is not affected.

Dunder-methods

Dunder-methods

Dunder-methods are special methods that are called by python in certain situations.

We have seen the __init__ method, which is called when a new object is created, before.

Further examples include:

- __str__ method is called when we try to convert an object to a string.
- __add__ method is called for + operator.
-

__lt__, __le__, __eq__, __ne__, __gt__, __ge__
methods used for comparisons.

Dunder-methods – Examples

One can now implement dunder methods for our custom classes to make certain functionality (like comparisons between objects) possible.

```
1     def __ge__(self, other):
2         if self.radius >= other.radius and self
3             .mass >= other.mass:
4             return True
5             return False
6
7     def __lt__(self, other):
8         return False if self.__ge__(other) else
9         True
```

Dunder-methods – Iterators

`__iter__` and `__next__` are special methods that allow us to iterate over objects.

While `__iter__` returns an iterator object, `__next__` returns the next element of the iterator.

This allows us to specify what a while/for loop should do when it encounters an object of our class.

Dunder-methods – Iterators – Example

Let's define a new class:

```
1 class Animal:
2     def __init__(self, name, type, age):
3         self.name = name
4         self.type = type
5         self.age = age
6
7     def __str__(self) -> str:
8         return f"{self.name} is {self.age} old
          and a {self.type}!"
```

...

Dunder-methods – Iterators – Example

```
1 class AnimalList:
2     def __init__(self):
3         self.animals = []
4         self.index = 0
5
6     def __add__(self, animal: Animal):
7         if animal not in self.animals:
8             self.animals.append(animal)
9
10    def __iter__(self):
11        self.index = 0
12        return self
```

...

Dunder-methods – Iterators – Example

```
1     def __next__(self):
2         while self.index < len(self.animals):
3             self.index += 1
4             return self.animals[self.index-1].
5         __str__()
6         raise StopIteration
7
8 joe = Animal("Joe", "Dog", 5)
9 bob = Animal("Bob", "Cat", 3)
10 my_animals = Animallist()
11 my_animals + joe # add animals to list
12 my_animals + bob
13 for animal in my_animals:
14     print(animal)
```

Inheritance

Inheritance

Inheritance is a way to create a new class from an existing class.

The new class is called child class and the existing class is called parent class.

The child class inherits all attributes and methods from the parent class and may expand on them or overwrite them.

Inheritance

We can create a child class by passing the parent class as an argument to the child class definition:

```
1 class Dog(Animal):
```

Here the child class Dog is instantiated with the parent class Animal as an argument.

```
1 class Animal():  
2     def __init__(self, name, type, age):  
3         self.name = name  
4         self.type = type  
5         self.age = age
```

Animal implements a `__str__` method like seen before.

Inheritance – super method

The super method allows us to call methods of the parent class.

This way we can call `__init__` in the parent class from the child class:

```
1 class Cat(Animal):  
2     def __init__(self, name, age):
```

Inheritance – Example

```
1 class Cat(Animal):
2     def __init__(self, name, age):
3         super().__init__(name, "Cat", age)
4
5     def make_sound(self):
6         print("Meow!")
7
8 class Dog(Animal):
9     def __init__(self, name, age):
10        super().__init__(name, "Dog", age)
11
12    def make_sound(self):
13        print("Woof!")
```

...

Inheritance – Example

```
1 dogo = Dog("Dogo", 5)
2 cat = Cat("Cat", 3)
3 dogo.make_sound()    # Woof!
4 cat.make_sound()     # Meow!
```

Now the method `make_sound` is implemented in both children classes.

Cats make a meow sound while dogs make a woof sound.

Inheritance – Overwriting methods

Ok let's add another child class, a frog:

```
1 class Frog(Animal):  
2     def __init__(self, name, age):  
3         super().__init__(name, "Frog", age)
```

We should also add a `make_sound` method to the parent class:

```
1     def make_sound(self):  
2         print("I am an animal!")
```

Inheritance – Overwriting methods

If we now instantiate a frog and call `make_sound` we get:

```
1 my_frog = Frog("Frog", 1)
2 my_frog.make_sound()    # I am an animal!
```

Because there is no `make_sound` method in the frog class, the method from the parent class is used.

The Cat and Dog class overwrite the method from the parent class and use their own implementation.

Wrap up + What's next?

Wrap up

By now you should have a good understanding of the basics of python programming for scientific computing.

You have learned how to use python to:

- import data
- modify and analyse the data
- plot/visualize the data
- write your own functions
- define classes and objects ...

What's next?

There are many more topics to cover in (python) programming ...

Next up: Computational Physics in 5th semester!

- numerical methods for solving differential equations
- methods for solving linear systems
- solving partial differential equations
- efficient eigenvalues of matrices / compression

What's next?

Other courses you might be interested in:

- Einführung in die strukturierte Programmierung (C)
[INB.04000UF, INB.05000UF]
- Objektorientierte Programmierung 1 (C++)
[INF.07002UF, INF.08008UF]
- Data Management (SQL) [INF.01017UF,
INF.02018UF]
- Datenstrukturen und Algorithmen 1 [INF.04032UF,
INF.03031UF]
- Software Engineering in Physics [PHT.528UF]
- ...